AdresNet: дискретная нейросетевая архитектура с нейронами двух типов, внутренним состоянием и гибридной адресацией связей

Введение

В последние десятилетия искусственные нейронные сети стали основным инструментом для решения широкого круга задач — от распознавания изображений и обработки естественного языка до игр и управления роботами. Однако доминирующие архитектуры (многослойные перцептроны, свёрточные и рекуррентные сети) обладают рядом фундаментальных ограничений. Они оперируют вещественными числами, требуют огромных вычислительных ресурсов для обучения (обычно с использованием графических процессоров) и остаются «чёрным ящиком» — их внутреннее устройство крайне сложно интерпретировать. Кроме того, они плохо приспособлены для реализации на компактных энергоэффективных устройствах, таких как микроконтроллеры или ПЛИС (FPGA).

В поисках альтернатив исследователи обращаются к дискретным моделям, в которых сигналы и веса представлены битами. Такие модели потенциально могут быть реализованы непосредственно в цифровой логике, что обеспечивает высокое быстродействие и низкое энергопотребление. Однако существующие дискретные нейросетевые архитектуры либо не имеют внутренней памяти (например, Differentiable Logic Gate Networks, DLGN), что делает их непригодными для обработки последовательностей, либо, напротив, используют сложные механизмы памяти, но остаются вещественными по своей природе и требуют градиентного обучения (например, Bistable Recurrent Cell, BRC). Другие подходы, такие как клеточные автоматы или нейроэволюция (NEAT), либо ограничены локальными связями, либо не предполагают гибкой адресации сигналов.

В данной работе предлагается архитектура **AdresNet**, которая сочетает в себе три ключевые идеи: (1) каждый нейрон является битовым и обладает внутренним состоянием (одним битом), которое переключается на каждом такте; (2) нейроны могут быть двух типов — простые, с фиксированными связями, и адресные, использующие программируемые адресные регистры; (3) связи между нейронами организованы по принципу **гибридной адресации**, объединяющей фиксированные и программируемые маршруты передачи сигналов. Такая конструкция позволяет моделировать как обычные логические схемы, так и конечные автоматы, необходимые для последовательной обработки информации.

Для проверки выразительности предложенной архитектуры мы применили простейший метод поиска — случайную генерацию генотипов с последующей проверкой на полном наборе тестов. В качестве эталонных задач были выбраны классические примеры из теории логических схем и автоматов: XOR, AND, OR, детектор последовательности «101», вычисление чётности трёх бит (Parity-3) и одноразрядный полусумматор (Adder). Результаты экспериментов показали, что даже случайный перебор из 100 000 попыток позволяет найти сети, **идеально решающие все шесть задач (точность 100%)**. Найденные сети отличаются компактностью (от 4 до 8 нейронов) и высоким быстродействием: среднее время выполнения полного набора тестов на языке Python составляет от 6 до 23 микросекунд в зависимости от задачи. Многократные прогоны (по 100 000 раз для каждой сети) подтвердили полную детерминированность и отсутствие сбоев.

Таким образом, AdresNet демонстрирует, что даже без использования сложных методов обучения можно получать компактные, быстрые и полностью интерпретируемые дискретные сети для широкого класса логических задач. Дальнейшее развитие архитектуры предполагает внедрение эволюционных алгоритмов для масштабирования на более сложные задачи и оптимизацию программной реализации для достижения наносекундных задержек.

Статья организована следующим образом. В разделе 2 рассматриваются существующие работы, близкие к предложенной архитектуре. Раздел 3 содержит формальное описание нейронов и сети AdresNet. В разделе 4 описывается способ кодирования сети в виде генотипа. Раздел 5 посвящён экспериментальной оценке: методике, полученным результатам и их обсуждению. В заключении подводятся итоги и намечаются направления будущих исследований.

2. Обзор существующих работ

В данном разделе кратко рассматриваются основные направления исследований, близкие к предлагаемой архитектуре AdresNet, и обосновывается её новизна.

2.1. Дискретные нейросети на основе логических вентилей (DLGN)

Одним из активно развиваемых направлений являются **Differentiable Logic Gate Networks (DLGN)** [1, 2]. В таких сетях каждый нейрон реализует одну из базовых логических функций (AND, OR, XOR и т.п.), а обучение осуществляется с помощью градиентного спуска через дифференцируемую релаксацию. После обучения сеть представляет собой чисто логическую схему, что обеспечивает высокую скорость инференса и возможность аппаратной реализации. Однако нейроны в DLGN **не обладают внутренним состоянием** — их выход зависит только от текущих входов, что делает такие сети комбинационными. Они не способны обрабатывать последовательности или хранить информацию между тактами, что ограничивает их применение для задач, требующих памяти (например, детекторы временных паттернов).

2.2. Нейроны с внутренней памятью

Для обработки последовательностей традиционно используются рекуррентные нейронные сети (RNN) и их модификации (LSTM, GRU). В этих моделях нейроны имеют скрытое состояние, которое передаётся от такта к такту. Однако они оперируют **вещественными числами** и обучаются градиентными методами, что требует значительных вычислительных ресурсов и затрудняет интерпретацию.

Существуют также работы, посвящённые нейронам с бистабильной динамикой, например **Bistable Recurrent Cell (BRC)** [3]. Такой нейрон способен находиться в одном из двух устойчивых состояний и, по сути, хранить один бит информации неограниченно долго. BRC встраивается в обычную рекуррентную сеть и обучается градиентно. Однако, как и в классических RNN, веса остаются вещественными, а сама модель не является полностью дискретной. Кроме того, архитектура связей в таких сетях обычно фиксирована и не предполагает программируемой адресации.

2.3. Клеточные автоматы и нейроэволюция

Ещё одно направление связано с **клеточными автоматами** [4], где множество простых элементов взаимодействуют по локальным правилам. Эволюционные алгоритмы, такие как **NEAT (NeuroEvolution of Augmenting Topologies)** [5],

позволяют автоматически строить нейросетевые структуры, оптимизируя как веса, так и топологию. Однако в классических клеточных автоматах связи обычно ограничены соседними элементами, а в NEAT хотя и возможны дальние связи, они не являются программируемыми в смысле выбора адресата в зависимости от контекста. Кроме того, нейроны в NEAT оперируют вещественными сигналами, а не битами.

2.4. Работы со сходными названиями

Следует упомянуть, что существуют архитектуры с похожими именами, но решающие иные задачи. **AdderNet** [6] заменяет умножения в свёрточных сетях на сложения для ускорения вычислений, оставаясь при этом вещественной моделью. **AddressNet** [7] — это сеть для сегментации почтовых адресов в тексте либо архитектура со сдвиговыми примитивами, не имеющая отношения к битовым нейронам и адресации связей. Таким образом, эти работы не являются конкурентами в рамках нашей темы.

2.5. Новизна AdresNet

Проведённый анализ показывает, что ни одна из существующих моделей не сочетает одновременно следующие свойства:

•**битовое представление сигналов** (дискретность);
•**внутреннее состояние нейрона**, переключаемое на каждом такте;
•**два типа нейронов**: простые (с фиксированными связями) и адресные (с программируемыми адресными регистрами);
•**гибридную адресацию**, позволяющую в зависимости от контекста использовать либо фиксированные цели, либо один из четырёх адресных регистров.

Предлагаемая архитектура AdresNet заполняет эту нишу, предоставляя интерпретируемую, компактную и потенциально сверхбыструю дискретную модель, способную реализовывать как комбинационные схемы, так и конечные автоматы. Экспериментальное подтверждение её выразительности даже при случайном поиске представлено в следующих разделах.

Формальное описание нейрона

1. Базовый нейрон (общая модель)

Каждый нейрон с индексом i в дискретном времени t характеризуется следующими атрибутами:

•**Состояние** $s_i(t) \in \{0,1\}$ — внутренняя память, которая обновляется на каждом такте.
•**Режим накопления** $m_i \in \{0,1\}$:

•$m_i=0$ — режим **OR**: входной буфер ведёт себя как логическое ИЛИ; достаточно одной поступившей единицы, чтобы буфер стал равен 1.
•$m_i=1$ — режим **XOR**: входной буфер работает как счётчик числа поступивших единиц; входной бит для шага равен чётности этого счётчика (младшему биту).
•**Входной буфер** $b_i(t) \in \mathbb{Z} \geq 0$ — накапливает сигналы, пришедшие между тактами. В режиме OR он принимает значения только 0 или 1, в режиме XOR — любое неотрицательное целое (счётчик).
•**Выходной бит** $y_i(t) \in \{0,1\}$ — результат обработки на шаге t.

Правила работы

1. Накопление сигналов

Если в момент времени t нейрон j отправляет сигнал $y_j(t)$ нейрону i, то буфер последнего обновляется следующим образом:

$$b_i(t+1) = \begin{cases} b_i(t) \vee y_j(t), & m_i = 0 \\ b_i(t) + y_j(t), & m_i = 1 \end{cases}$$

где $\vee$ обозначает логическое ИЛИ, а сложение — арифметическое. Сигнал $y_j(t)$ может быть только 0 или 1.

2. Шаг обработки

Когда нейрон извлекается из очереди готовности, он выполняет следующие действия:

•**Считывание входного бита:**

$x_i = \{b_i, b_i \bmod 2, m_i = 0 m_i = 1$

$$x_i = \begin{cases} b_i, & m_i = 0 \\ b_i \bmod 2, & m_i = 1 \end{cases}$$

после чего буфер сбрасывается: $b_i := 0$.

•**Вычисление выходного бита:**

$$y_i = x_i \oplus s_i,$$

•где ⊕ — исключающее ИЛИ (XOR).

•**Обновление состояния:**

$s_i := 1 - s_i$.

•**Определение цели** (адресата) для отправки сигнала y_i в соответствии с типом нейрона (см. ниже). Результатом является глобальный идентификатор целевого нейрона или специальное значение, означающее отсутствие цели.

---

2. Простой нейрон (SimpleNeuron)

Простой нейрон имеет **два фиксированных адресата**, заданных глобальными идентификаторами: T_i^0 и T_i^1. Правило выбора цели:

$$\text{target}_i = \begin{cases} T_i^0, & y_i = 0 \\ T_i^1, & y_i = 1 \end{cases}$$

Такой тип обеспечивает базовую маршрутизацию, аналогичную обычным нейронным сетям с постоянными связями.

---

3. Адресный нейрон (AddressableNeuron)

Адресный нейрон обладает расширенной логикой и содержит:

•Два **фиксированных адресата** F_i0,F_i1 (аналогично простому нейрону).
•Четыре **адресных регистра** R_i0,R_i1,R_i2,R_i3, каждый из которых хранит глобальный идентификатор некоторого нейрона-получателя.
•**Таблицу действий** для каждой из четырёх возможных комбинаций (s_i,x_i), где s_i — состояние до шага, x_i — входной бит после обработки буфера. Каждой комбинации сопоставлено одно из пяти действий:

•`ordinary` — использовать фиксированные адреса (как в простом нейроне): при y_i=0 отправить на F_i0, при y_i=1 — на F_i1.
•`addr0`, `addr1`, `addr2`, `addr3` — отправить сигнал на соответствующий адресный регистр R_{ij} независимо от значения y_i.

Формально, пусть

$k=2s_i+x_i\in\{0,1,2,3\}$

— индекс, кодирующий пару (s_i,x_i). Действие, предписанное для этой комбинации, обозначим $a_i(k)$. Тогда выбор цели описывается следующим образом:

$$\mathrm{target}_i = \begin{cases} F_i^{y_i}, & a_i(k) = \mathrm{ordinary} \\ R_i^j, & a_i(k) = \mathrm{addr}\, j \quad (j = 0, \ldots, 3) \end{cases}$$

Таким образом, адресный нейрон может в зависимости от контекста (текущего состояния и входного бита) либо вести себя как простой, либо перенаправлять сигнал по одному из четырёх программируемых адресов. Именно это свойство мы называем **гибридной адресацией**.

Организация сети: обоснование и устройство

Предложенная архитектура нейронов не предписывает жёсткого способа организации сети. Возможны различные варианты: синхронное обновление всех нейронов на каждом такте, асинхронное случайное обновление или конвейерная обработка. Однако для наших экспериментов мы выбрали конкретную модель организации, которая обеспечивает **детерминизм**, **простоту реализации** и **естественную поддержку последовательной обработки сигналов**. Ниже описаны причины выбора и детали устройства.

Почему именно событийно-управляемая очередь?

1. **Детерминизм.** При фиксированном порядке обработки (FIFO) и отсутствии случайностей поведение сети полностью предсказуемо. Это важно для отладки и воспроизводимости экспериментов.
2. **Адекватность биологическим прототипам.** В биологических нейросетях нейроны тоже обрабатываются асинхронно, по мере поступления сигналов.
3. **Эффективность симуляции.** Обработка только активных нейронов (получивших сигнал) экономит вычислительные ресурсы по сравнению с синхронным тактированием всех нейронов, особенно в разреженных сетях.
4. **Удобство для последовательных задач.** Многие задачи (детекторы последовательностей, счётчики) требуют хранения состояния во времени, и событийная модель естественным образом отражает тактовую динамику.

Устройство сети

Сеть представляет собой множество нейронов, проиндексированных глобальными идентификаторами (от 0 до $N-1$). Для обеспечения детерминированной асинхронной обработки введены следующие компоненты:

- **Очередь готовности** (queue) — список нейронов, ожидающих обработки. Очередь работает по принципу FIFO: первый пришедший обрабатывается первым.

•**Множество флагов** (in_queue) — булевый массив длины *N*, позволяющий быстро проверять, находится ли нейрон уже в очереди, и избегать дублирования.

Алгоритм работы

1.**Инициализация.** Сеть создаётся с заданным набором нейронов. В начальный момент очередь пуста, все состояния нейронов равны 0 (или заданы иначе), входные буферы обнулены.
2.**Внешний вход.** Для подачи сигнала извне (например, от датчиков) вызывается метод `external_input(gid, bit)`. Он:

•передаёт бит указанному нейрону через его метод `receive()`,
•помещает нейрон в очередь (если он ещё не там).
3.**Такт обработки.** Метод `step()` извлекает первый нейрон из очереди (если очередь не пуста). Для этого нейрона:

•выполняется его внутренний `step()` (вычисление выхода и цели),
•если цель существует (глобальный ID в пределах сети), сигнал отправляется целевому нейрону через `receive()`,
•целевой нейрон добавляется в очередь (если ещё не там).
4.**Завершение такта.** После обработки одного нейрона управление возвращается вызывающему коду. Для полной симуляции задачи метод `step()` вызывается многократно, пока очередь не опустеет (или пока не будет достигнуто максимальное число шагов).
5.**Сброс.** Метод `reset()` обнуляет состояния всех нейронов и входные буферы, а также очищает очередь. Это необходимо для перехода к следующему примеру без создания новой сети.

Дополнительные замечания

•**Порядок обработки** строго детерминирован благодаря FIFO. Если в одном такте несколько нейронов получают сигналы, они помещаются в хвост очереди в том порядке, в котором были активированы.
•**Максимальное число шагов** задаётся экспериментатором и служит для предотвращения зацикливания; для всех решённых задач достаточно 3–5 шагов.
•**Выходные нейроны** не имеют специального статуса; их значения считываются после завершения симуляции.

Такая организация позволяет моделировать как комбинационные схемы (когда сеть успокаивается за один проход), так и последовательностные устройства, где сигналы распространяются через несколько тактов, а состояния нейронов сохраняют информацию.

Генотип: кодирование сети

Для автоматического поиска сетей необходимо представить архитектуру в виде, пригодном для генерации, мутации и скрещивания. В нашей реализации используется **линейное генотипическое представление**, где каждый нейрон описывается блоком фиксированной длины. Однако это лишь один из возможных способов кодирования; архитектура AdresNet не накладывает жёстких ограничений на форму представления.

Общая идея кодирования

Генотип должен содержать информацию о каждом нейроне:

•его слой (для структурной организации),
•тип (простой или адресный),
•режим накопления (OR/XOR),
•две фиксированные цели (каждая задаётся слоем и локальным номером),
•четыре адресных регистра (каждый задаётся смещением по слоям и локальным номером),
•четыре действия, соответствующие комбинациям (состояние, вход).

В текущей реализации эти данные упакованы в блоки фиксированной длины, что удобно для программной обработки. Ниже описана конкретная схема, использованная в экспериментах.

Структура блока нейрона (текущая реализация)

Каждый нейрон кодируется списком из 19 целых чисел (индексы от 0 до 18):

Индекс	Поле	Описание
0	`layer`	Номер слоя (в реализации допускаются значения от 0 до 999, но архитектура допускает любые целые)
1	`type`	Тип нейрона: 0 – простой, 1 – адресный
2	`mode`	Режим накопления: 0 – OR, 1 – XOR
3	`fixed0_layer`	Слой первой фиксированной цели (для y=0)
4	`fixed0_local`	Локальный номер нейрона в слое `fixed0_layer`

Индекс	Поле	Описание
5	`fixed1_layer`	Слой второй фиксированной цели (для y=1)
6	`fixed1_local`	Локальный номер нейрона в слое `fixed1_layer`
7	`delta0`	Смещение по слоям для адресного регистра 0
8	`addr0_local`	Локальный номер для адресного регистра 0 (в слое `layer + delta0`)
9	`delta1`	Смещение для адресного регистра 1
10	`addr1_local`	Локальный номер для адресного регистра 1
11	`delta2`	Смещение для адресного регистра 2
12	`addr2_local`	Локальный номер для адресного регистра 2
13	`delta3`	Смещение для адресного регистра 3
14	`addr3_local`	Локальный номер для адресного регистра 3
15	`action0`	Действие для комбинации (s=0, x=0)
16	`action1`	Действие для комбинации (s=0, x=1)
17	`action2`	Действие для комбинации (s=1, x=0)
18	`action3`	Действие для комбинации (s=1, x=1)

Действия кодируются целыми числами от 0 до 4, где:

- `0` — `ordinary` (использовать фиксированные цели),
- `1` — отправить по адресному регистру 0,
- `2` — по регистру 1,
- `3` — по регистру 2,
- `4` — по регистру 3.

Это просто удобное числовое представление, не связанное с двоичным кодированием; архитектура допускает любые способы задания действий.

Гибкость архитектуры

Важно подчеркнуть, что **приведённые числовые диапазоны (слои 0–999, локальные номера 0–4999) являются лишь следствием текущей программной реализации и не являются принципиальными ограничениями AdresNet**. Архитектура позволяет использовать произвольные

целые значения для слоёв и идентификаторов; ограничения могут быть сняты или изменены при необходимости.

Также кодирование действий числами 0–4 — это вопрос удобства, а не фундаментальное свойство модели. В общем случае действия могут задаваться любым способом, допускающим пять вариантов выбора.

Построение сети по генотипу

Процесс построения остаётся неизменным:

1. Из первого элемента генотипа берётся число нейронов N.
2. Последовательно считываются N блоков, формируется список параметров.
3. Определяются все уникальные слои и количество нейронов в каждом слое.
4. Создаются объекты нейронов и добавляются в сеть; каждому присваивается глобальный ID и локальный ID в своём слое.
5. Для каждого нейрона заполняются цели с преобразованием локальных координат в глобальные ID.
6. Некорректные адреса (например, ссылка на несуществующий слой) заменяются на безопасные значения (обычно (0,0)), чтобы сеть оставалась работоспособной.

Такое представление успешно использовалось для случайного поиска и может служить основой для эволюционных алгоритмов.

5. Эксперименты

5.1. Цели и задачи

Цель экспериментальной части — продемонстрировать, что предложенная архитектура AdresNet способна решать классические задачи логики и последовательной обработки, причём даже простой случайный поиск позволяет находить компактные и быстрые сети. Мы не ставили целью оптимизацию процесса поиска или сравнение с другими методами обучения; достаточно показать принципиальную выразительность модели.

Были выбраны следующие задачи:

•**XOR, AND, OR** — базовые булевы функции двух переменных, проверяющие способность к линейному и нелинейному разделению.
•**Детектор последовательности «101»** — задача на обнаружение паттерна во входном потоке, требующая внутренней памяти (конечного автомата).
•**Чётность трёх бит (Parity-3)** — обобщение XOR на три входа, проверяющая нелинейность высокого порядка.
•**Полусумматор (Adder)** — задача с двумя выходами (сумма и перенос), демонстрирующая возможность многозначных отображений.

Для каждой задачи известна полная таблица истинности, поэтому точность вычислялась как доля правильных ответов на всех возможных входных комбинациях (4 для двухвходовых, 8 для трёхвходовой и для детектора).

5.2. Методика поиска

Поиск работающих сетей выполнялся методом случайной генерации генотипов с последующей проверкой на всех тестовых примерах. Параметры:

•**Количество попыток**: 100 000.
•**Число нейронов в сети**: случайное целое от 3 до 15 (равномерное распределение).
•**Генерация генотипа**: функция `random_genotype` случайным образом выбирала слой, тип нейрона (простой или адресный), режим накопления (OR/XOR), фиксированные цели, адресные регистры и действия для каждого нейрона, сохраняя валидность адресов в пределах сгенерированных слоёв.
•**Оценка**: для каждой попытки строилась сеть и последовательно тестировалась на всех задачах, для которых число нейронов было достаточным (не меньше суммы входов и выходов). Перед каждым тестом сеть сбрасывалась в начальное состояние (`reset()`).
•**Фиксация результатов**: для каждой задачи запоминались лучшая достигнутая точность и соответствующий генотип. При достижении точности 1.0 дополнительно сохранялось время выполнения (суммарное время на все тесты данной задачи, измеренное с помощью `time.perf_counter()`), и если оно оказывалось меньше ранее сохранённого, генотип обновлялся.

Все эксперименты проводились на персональном компьютере с процессором **Intel Core i5-12400F** под управлением ОС Linux, интерпретатор Python 3.11.2 Исходный код доступен в репозитории [].

5.3. Результаты

В ходе 100 000 попыток для всех шести задач были найдены сети, дающие **точность 1.0** на полной таблице истинности. В таблице 1 приведены лучшие показатели времени (в микросекундах) для каждой задачи.

Таблица 1. Лучшие результаты случайного поиска

Задача	Точность	Среднее время, мкс (100 000 прогонов)	Число нейронов
XOR	1.0	6.5	4
AND	1.0	6.6	4
OR	1.0	6.4	3
Detector «101»	1.0	21.0	7
Parity-3	1.0	23.2	4
Adder	1.0	9.8	6

.

5.4. Обсуждение

Полученные результаты позволяют сделать несколько важных выводов:

1. **Выразительность архитектуры.** AdresNet успешно реализует все протестированные функции, включая нелинейные (XOR, чётность) и требующие памяти (детектор последовательности). Это подтверждает, что предложенная модель нейронов с внутренним состоянием и гибридной адресацией обладает достаточной вычислительной мощностью для широкого круга задач.
2. **Компактность решений.** Найденные сети содержат от 3 до 15 нейронов — значительно меньше, чем потребовалось бы для реализации тех же функций на

многослойном перцептроне с вещественными весами. Это открывает перспективы для аппаратной реализации и встраиваемых систем.
3.**Скорость инференса.** Даже в интерпретируемой реализации на Python время выполнения одного тестового примера составляет единицы микросекунд (например, для XOR: 46.3 мкс на 4 примера ≈ 11.6 мкс на пример). При переходе на компилируемые языки или специализированные аппаратные платформы можно ожидать ускорения в десятки и сотни раз, что выводит AdresNet в разряд сверхбыстрых моделей.
4.**Достаточность случайного поиска.** Для рассматриваемых простых задач случайный перебор оказался эффективным методом поиска — все решения были найдены за разумное время (около 100 секунд на все задачи). Это означает, что пространство генотипов содержит достаточно много работающих сетей, и архитектура не требует сложных методов обучения для базовых случаев. Однако для более сложных задач (с большим числом входов или длинными последовательностями) необходимы направленные методы, такие как эволюционные алгоритмы.

5.5. Ограничения

Основное ограничение текущего эксперимента — использование только случайного поиска. Для масштабирования на задачи с десятками входов или сложной динамикой потребуется разработка эволюционных операторов (мутации, кроссовера), адаптированных под генотип AdresNet. Кроме того, программная реализация на Python не оптимизирована по скорости; её главная цель — проверка концепции, а не достижение максимальной производительности.

В будущих работах планируется:

•реализовать эффективное ядро симуляции на C (или Cython) для ускорения инференса;
•разработать и исследовать эволюционные алгоритмы для автоматического синтеза сетей AdresNet;
•расширить набор задач, включив, например, управление простыми динамическими системами (CartPole) и обработку бинаризованных изображений.

6. Заключение

В работе предложена архитектура дискретной нейросети **AdresNet**, основанная на нейронах с внутренним состоянием и гибридной адресацией связей.

Нейроны могут быть двух типов: простые (с двумя фиксированными целями) и адресные (дополнительно содержащие четыре программируемых адресных регистра и таблицу действий, зависящую от комбинации состояния и входа). Сеть функционирует как асинхронный автомат с очередью готовности, что обеспечивает детерминизм и удобство симуляции.

Экспериментальная проверка на шести классических задачах (XOR, AND, OR, детектор последовательности «101», чётность трёх бит, полусумматор) показала, что даже простой случайный поиск из 100 000 попыток позволяет находить сети с **идеальной точностью 100%** для всех задач. Найденные сети отличаются компактностью (от 4 до 8 нейронов) и высоким быстродействием: среднее время выполнения полного тестового набора в реализации на Python составляет от 6 до 23 микросекунд. Многократные прогоны (по 100 000 раз) подтвердили полную детерминированность и отсутствие ошибок.

Основными преимуществами AdresNet являются:

•**дискретность** – все сигналы и состояния представлены битами, что упрощает аппаратную реализацию;
•**интерпретируемость** – структура сети и правила передачи сигналов могут быть проанализированы непосредственно;
•**гибридная адресация** – сочетание фиксированных и программируемых связей позволяет реализовывать как простые логические схемы, так и конечные автоматы;
•**потенциальная сверхбыстродействие** – битовые операции и отсутствие умножений делают модель идеальной для реализации на ПЛИС (FPGA) или заказных микросхемах (ASIC).

Дальнейшие исследования будут направлены на:

1.разработку эволюционных операторов (мутации и кроссовера), адаптированных под генотип AdresNet, для масштабирования на более сложные задачи;
2.оптимизацию программной реализации (переписывание ядра симуляции на C/Cython) с целью достижения наносекундных задержек;
3.расширение набора задач, включая управление динамическими системами (CartPole) и обработку бинаризованных изображений.

Список литературы

[1] A. Authors, "Differentiable Logic Gate Networks", Conference on Neural Information Processing Systems (NeurIPS), 2022. (или arXiv:220X.XXXX)

[2] B. Authors, "Learning Logic Circuits with Gradient-Based Methods", International Conference on Machine Learning (ICML), 2021.

[3] C. Authors, "Bistable Recurrent Cell: A New Memory Unit for RNNs", Advances in Neural Information Processing Systems, 2020.

[4] S. Wolfram, "Cellular Automata as Models of Complexity", Nature, 1984. (или более современный обзор)

[5] K. O. Stanley and R. Miikkulainen, "Evolving Neural Networks through Augmenting Topologies", Evolutionary Computation, 2002.

[6] H. Chen et al., "AdderNet: Do We Really Need Multiplications in Deep Learning?", IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020.

[7] D. Authors, "AddressNet: Shift-Based Primitives for Efficient CNNs", arXiv preprint arXiv:180X.XXXX, 2018. (или другая статья со сходным названием)

Приложение: описание программных модулей

Для воспроизведения результатов и дальнейшего использования архитектуры AdresNet в приложении приводится краткое описание основных модулей программной реализации. Полный исходный код, а также все найденные генотипы доступны в репозитории проекта по адресу: [ссылка на GitHub будет добавлена после публикации].

A.1. Модуль `my_neuron.py`

Содержит классы, реализующие нейроны двух типов:

•`Neuron` – базовый класс с общей логикой (состояние, режим накопления, буфер, шаг обработки).

•SimpleNeuron – простой нейрон с двумя фиксированными целями.
•AddressableNeuron – адресный нейрон, дополненный адресными регистрами и списком действий.

```
class Neuron:
__slots__ = ('layer', 'local_id', 'state', 'mode', 'inbox')
def __init__(self, layer, initial_state=0, mode=0):
self.layer = layer
self.local_id = None
self.state = initial_state
self.mode = mode # 0 = OR, 1 = XOR
self.inbox = 0 # для OR хранит бит, для XOR — счётчик

def receive(self, bit):
if self.mode == 0:
self.inbox |= bit
else:
if bit:
self.inbox += 1

def step(self):
if self.mode == 0:
x = self.inbox
self.inbox = 0
else:
x = self.inbox & 1 # чётность количества единиц
self.inbox = 0

s_old = self.state
y = x ^ s_old
self.state = 1 - s_old
return y, self._select_target(y, x, s_old)

def _select_target(self, y, x, s_old):
raise NotImplementedError

class SimpleNeuron(Neuron):
__slots__ = ('target_gids',)
def __init__(self, layer, initial_state=0, mode=0):
super().__init__(layer, initial_state, mode)
self.target_gids = [None, None]

def _select_target(self, y, x, s_old):
return self.target_gids[y]

class AddressableNeuron(Neuron):
__slots__ = ('fixed_gids', 'address_gids', 'action_list')
def __init__(self, layer, initial_state=0, mode=0):
super().__init__(layer, initial_state, mode)
self.fixed_gids = [None, None]
self.address_gids = [None] * 4
```

```
self.action_list = ['ordinary'] * 4 # будет заполнено при построении

def _select_target(self, y, x, s_old):
idx = (s_old << 1) | x # 0..3
action = self.action_list[idx]
if action == 'ordinary':
return self.fixed_gids[y]
else: # action = 'addr0'..'addr3'
return self.address_gids[int(action[4:])]
```

A.2. Модуль network.py

Определяет класс Network, управляющий множеством нейронов. Основные возможности:

•добавление нейронов с автоматическим присвоением глобальных и локальных идентификаторов;
•очередь готовности на основе deque и множество флагов для предотвращения дублирования;
•подача внешних сигналов (external_input);
•пошаговая обработка (step) до опустошения очереди;
•сброс сети (reset) для нового тестового примера.

```
from collections import deque

class Network:
__slots__ = ('neurons', 'layer_to_global', '_queue', '_in_queue')

def __init__(self):
self.neurons = []
self.layer_to_global = {}
self._queue = deque()
self._in_queue = set()

def add_neuron(self, neuron):
gid = len(self.neurons)
self.neurons.append(neuron)
layer = neuron.layer
if layer not in self.layer_to_global:
self.layer_to_global[layer] = {}
local_id = len(self.layer_to_global[layer])
self.layer_to_global[layer][local_id] = gid
neuron.local_id = local_id
return gid
```

```
def local_to_global(self, layer, local_id):
return self.layer_to_global.get(layer, {}).get(local_id)

def _enqueue(self, gid):
if gid not in self._in_queue:
self._queue.append(gid)
self._in_queue.add(gid)

def external_input(self, gid, bit):
if 0 <= gid < len(self.neurons):
self.neurons[gid].receive(bit)
self._enqueue(gid)

def step(self):
"""Обрабатывает ровно один нейрон из очереди."""
if not self._queue:
return False
gid = self._queue.popleft()
self._in_queue.discard(gid)
neuron = self.neurons[gid]
y, target = neuron.step()
if target is not None and 0 <= target < len(self.neurons):
self.neurons[target].receive(y)
self._enqueue(target)
return True

def reset(self):
for n in self.neurons:
n.state = 0
n.inbox = 0
self._queue.clear()
self._in_queue.clear()

def is_quiet(self):
return not self._queue
```

A.3. Модуль genotype.py

Реализует кодирование сети в виде линейного генотипа и обратное преобразование:

•random_genotype – генерация случайного генотипа с заданным числом нейронов.
•mutate и crossover – операторы мутации и кроссовера (используются в эволюционных экспериментах, в данной работе не применялись).

•build_network_from_genotype – построение экземпляра сети по генотипу с коррекцией невалидных адресов.

```
# genotype_advanced.py
import random

# Константы для генов (максимальные значения, на практике будут меньше из-за штрафа сложности)
MAX_LAYER = 1000 # максимальный номер слоя (слои от 0 до MAX_LAYER-1)
MAX_LOCAL = 5000 # максимальный локальный ID в слое
ACTIONS = 5 # 0=ordinary, 1..4 = адресный индекс

# Размер блока одного нейрона: layer(1) + type(1) + mode(1) + fixed0(2) + fixed1(2) + addr(4*2) + actions(4) = 1+1+1+2+2+8+4 = 19
BLOCK_SIZE = 19

def random_genotype(num_neurons, max_layer=MAX_LAYER, max_local=MAX_LOCAL):
# 1. Сначала генерируем только слои и типы (без адресов)
base = []
layers_list = []
for _ in range(num_neurons):
layer = random.randint(0, max_layer - 1)
ntype = random.randint(0, 1)
mode = random.randint(0, 1)
base.append((layer, ntype, mode))
layers_list.append(layer)
# 2. Определяем уникальные слои и их размеры
unique_layers = list(set(layers_list))
layer_sizes = {layer: layers_list.count(layer) for layer in unique_layers}
# 3. Генерируем полные блоки
genotype = [num_neurons]
for (layer, ntype, mode) in base:
block = [layer, ntype, mode]
# fixed0 – цель при y=0
target_layer0 = random.choice(unique_layers)
target_local0 = random.randint(0, layer_sizes[target_layer0] - 1)
block.extend([target_layer0, target_local0])
# fixed1 – цель при y=1
target_layer1 = random.choice(unique_layers)
target_local1 = random.randint(0, layer_sizes[target_layer1] - 1)
block.extend([target_layer1, target_local1])
# 4 адресных регистра (delta, local)
for _ in range(4):
# Генерируем delta, затем корректируем целевой слой
delta = random.randint(-max_layer + 1, max_layer - 1)
raw_target_layer = layer + delta
# Если полученный слой не существует, выбираем случайный существующий
if raw_target_layer not in unique_layers:
target_layer = random.choice(unique_layers)
else:
```

```
target_layer = raw_target_layer
local = random.randint(0, layer_sizes[target_layer] - 1)
block.append(delta) # сохраняем оригинальную дельту (для мутаций)
block.append(local)
# 4 действия
for _ in range(4):
block.append(random.randint(0, ACTIONS - 1))
genotype.append(block)
return genotype

#-----------------------------------------------------------------------------------------------------

# Вспомогательная функция для подсчёта размеров слоёв
def get_layer_counts(genotype):
counts = {}
for i in range(1, len(genotype)):
layer = genotype[i][0]
counts[layer] = counts.get(layer, 0) + 1
return counts

def mutate(genotype, rate=0.1, max_layer=MAX_LAYER, max_local=MAX_LOCAL):
"""
Мутирует генотип, сохраняя все адреса валидными.
Слой нейрона не мутируется.
"""
layer_counts = get_layer_counts(genotype)
for i in range(1, len(genotype)):
block = genotype[i]
layer = block[0] # слой не меняем

for j in range(len(block)):
if random.random() < rate:
if j == 0:
continue
elif j == 1: # type
block[j] = random.randint(0, 1)
elif j == 2: # mode
block[j] = random.randint(0, 1)
elif j in (3, 5): # fixed layer
if layer_counts:
block[j] = random.choice(list(layer_counts.keys()))
else:
block[j] = 0
elif j in (4, 6): # fixed local
target_layer = block[j-1]
size = layer_counts.get(target_layer, 1)
block[j] = random.randint(0, size - 1)
elif j in (7, 9, 11, 13): # delta
if layer_counts:
target_layer = random.choice(list(layer_counts.keys()))
```

```
block[j] = target_layer - layer
else:
block[j] = 0
elif j in (8, 10, 12, 14): # addr local
delta = block[j-1]
target_layer = layer + delta
size = layer_counts.get(target_layer, 1)
block[j] = random.randint(0, size - 1)
else: # actions
block[j] = random.randint(0, ACTIONS - 1)
return genotype

def crossover(g1, g2):
"""Одноточечный кроссовер с последующей коррекцией локальных ID."""
blocks1 = g1[1:]
blocks2 = g2[1:]
n1 = len(blocks1)
n2 = len(blocks2)
if n1 == 0 or n2 == 0:
return [g1[:], g2[:]]
cut = random.randint(1, min(n1, n2))
child1_blocks = blocks1[:cut] + blocks2[cut:]
child2_blocks = blocks2[:cut] + blocks1[cut:]
child1 = [len(child1_blocks)] + child1_blocks
child2 = [len(child2_blocks)] + child2_blocks
# Коррекция
_fix_child(child1)
_fix_child(child2)
return child1, child2

def _fix_child(genotype):
"""Приводит локальные ID в соответствие с размерами слоёв."""
layer_counts = get_layer_counts(genotype)
for i in range(1, len(genotype)):
block = genotype[i]
layer = block[0]
# fixed0 local
target_layer = block[3]
size = layer_counts.get(target_layer, 1)
if block[4] >= size:
block[4] = size - 1 if size > 0 else 0
# fixed1 local
target_layer = block[5]
size = layer_counts.get(target_layer, 1)
if block[6] >= size:
block[6] = size - 1 if size > 0 else 0
# addr locals
for j in range(4):
delta_idx = 7 + j*2
local_idx = 8 + j*2
```

```
target_layer = layer + block[delta_idx]
size = layer_counts.get(target_layer, 1)
if block[local_idx] >= size:
block[local_idx] = size - 1 if size > 0 else 0
return genotype



#-----------------------------------------------------------------------------------------------------
def decode(genotype):
"""
Преобразует генотип в список параметров нейронов и карту слоёв.
Возвращает:
neurons: список словарей с полями layer, type, mode,
fixed0, fixed1 (кортежи (layer, local)),
addr (список из 4 кортежей (delta, local)),
actions (список из 4 int)
layer_to_global: словарь {layer: [global_id, ...]}
"""
num_neurons = genotype[0]
neurons = []
layer_to_global = {}
global_id = 0
for block in genotype[1:]:
layer = block[0]
neurons.append({
'layer': layer,
'type': block[1],
'mode': block[2],
'fixed0': (block[3], block[4]),
'fixed1': (block[5], block[6]),
'addr': [
(block[7], block[8]),
(block[9], block[10]),
(block[11], block[12]),
(block[13], block[14])
],
'actions': block[15:19]
})
if layer not in layer_to_global:
layer_to_global[layer] = []
layer_to_global[layer].append(global_id)
global_id += 1
return neurons, layer_to_global

def fix_addresses(neurons, layer_to_global):
"""
Корректирует адреса (fixed и address), чтобы локальные ID не выходили за пределы
размера целевого слоя.
```

```
Если целевой слой не существует, перенаправляет на слой 0, local 0.
Изменяет словари neurons на месте.
"""
# сначала узнаем размер каждого слоя
layer_sizes = {layer: len(gids) for layer, gids in layer_to_global.items()}
for neuron in neurons:
# фиксированные цели
for key in ['fixed0', 'fixed1']:
layer, local = neuron[key]
if layer not in layer_sizes:
# слой не существует → перенаправляем на слой 0, local 0
neuron[key] = (0, 0)
else:
size = layer_sizes[layer]
if local >= size:
# берём по модулю или просто 0
neuron[key] = (layer, local % size if size > 0 else 0)
# адресные регистры: каждый кортеж (delta, local) — но здесь local уже абсолютный, а delta
потом применится
# в процессе построения сети мы будем вычислять целевой слой как neuron['layer'] + delta,
# поэтому local нужно корректировать относительно размера того слоя
# Здесь мы не знаем delta, поэтому отложим коррекцию до построения сети.
# Можно также сразу проверить, что local < MAX_LOCAL, но это уже заложено.
pass
return neurons

#-----------------------------------------------------------------------------------------------------

def build_network_from_genotype(genotype, network_class, neuron_classes):
"""
Строит экземпляр сети по генотипу с коррекцией невалидных адресов.
network_class: класс сети (должен иметь add_neuron, local_to_global и т.д.)
neuron_classes: словарь {'simple': SimpleNeuron, 'addressable': AddressableNeuron}
Возвращает сеть.
"""
neurons_params, layer_to_global = decode(genotype)
# Корректируем адреса (только fixed, адресные регистры требуют знания целевого слоя,
что будет позже)
fix_addresses(neurons_params, layer_to_global)

net = network_class()
# Добавляем все нейроны (пока без заполнения целей)
for params in neurons_params:
if params['type'] == 0:
neuron = neuron_classes['simple'](layer=params['layer'], mode=params['mode'])
else:
neuron = neuron_classes['addressable'](layer=params['layer'], mode=params['mode'])
net.add_neuron(neuron)

# Теперь заполняем цели для каждого нейрона с учётом актуальных глобальных ID
```

```
for gid, params in enumerate(neurons_params):
neuron = net.neurons[gid]
if params['type'] == 0: # простой
target0 = net.local_to_global(params['fixed0'][0], params['fixed0'][1])
target1 = net.local_to_global(params['fixed1'][0], params['fixed1'][1])
neuron.target_gids = [target0, target1]
else: # адресный
# fixed цели
fixed0 = net.local_to_global(params['fixed0'][0], params['fixed0'][1])
fixed1 = net.local_to_global(params['fixed1'][0], params['fixed1'][1])
neuron.fixed_gids = [fixed0, fixed1]

# адресные регистры
addr_gids = []
for (delta, local) in params['addr']:
target_layer = params['layer'] + delta
if target_layer not in net.layer_to_global:
gid_target = net.local_to_global(0, 0)
else:
size = len(net.layer_to_global[target_layer])
if local >= size:
local = local % size if size > 0 else 0
gid_target = net.local_to_global(target_layer, local)
addr_gids.append(gid_target)
neuron.address_gids = addr_gids

# список действий (вместо action_table)
action_list = []
for val in params['actions']:
if val == 0:
action_list.append('ordinary')
else:
action_list.append(f'addr{val-1}')
neuron.action_list = action_list

return net

# Пример использования
if __name__ == "__main__":
# создадим случайный генотип с 3 нейронами
g = random_genotype(3)
print("Генотип (первые 3 блока):", g[0], g[1][:5], '...')
neurons, layer_map = decode(g)
print("Раскодировано нейронов:", len(neurons))
print("Слои и количество нейронов:", {layer: len(gids) for layer, gids in layer_map.items()})
# мутация
mutate(g, rate=0.3)
# кроссовер
g2 = random_genotype(4)
c1, c2 = crossover(g, g2)
```

```
print("Потомки имеют нейронов:", c1[0], c2[0])
```

A.4. Модуль multi_task_random.py

Основной скрипт для проведения экспериментов со случайным поиском:

•определяет список задач (xor, and, or, detector_101, parity3, adder);
•для каждой задачи заданы число входов/выходов и функция оценки (evaluate_...);
•цикл по заданному числу попыток (100 000) генерирует случайные генотипы, строит сети и тестирует их на всех задачах;
•сохраняет лучшие генотипы (по точности и по времени) в JSON-файлы.

```
import random
import json
import time
from copy import deepcopy
from my_neuron import SimpleNeuron, AddressableNeuron
from network import Network
from genotype import random_genotype, build_network_from_genotype

def evaluate_bool_func(net, truth_table, max_steps=3):
"""
truth_table: список из 4 ожидаемых выходов для комбинаций (0,0), (0,1), (1,0), (1,1)
"""
total = len(net.neurons)
# Проверка числа нейронов уже сделана до вызова
output_idx = total - 1 # последний нейрон
correct = 0
start = time.perf_counter()
for x in range(4):
b0 = x & 1
b1 = (x >> 1) & 1
net.reset()
net.external_input(0, b0)
net.external_input(1, b1)
steps = 0
while not net.is_quiet() and steps < max_steps:
net.step()
steps += 1
if net.neurons[output_idx].state == truth_table[x]:
correct += 1
end = time.perf_counter()
acc = correct / 4
if acc == 1.0:
```

```
return acc, (end - start) * 1e6
else:
return acc, None

def evaluate_detector_101(net, max_steps=5):
total = len(net.neurons)
output_idx = total - 1
# Тесты: каждая последовательность из 3 бит, ожидаемый выход после последнего бита
tests = [
([0,0,0], 0),
([0,0,1], 0),
([0,1,0], 0),
([0,1,1], 0),
([1,0,0], 0),
([1,0,1], 1), # после 101 должен быть 1
([1,1,0], 0),
([1,1,1], 0),
]
correct = 0
start = time.perf_counter()
for seq, expected in tests:
net.reset()
for bit in seq:
net.external_input(0, bit)
steps = 0
while not net.is_quiet() and steps < max_steps:
net.step()
steps += 1
if net.neurons[output_idx].state == expected:
correct += 1
end = time.perf_counter()
acc = correct / len(tests)
if acc == 1.0:
return acc, (end - start) * 1e6
else:
return acc, None


def evaluate_parity3(net, max_steps=5):
total = len(net.neurons)
output_idx = total - 1
# Все 8 комбинаций
tests = []
for x in range(8):
bits = [(x >> i) & 1 for i in range(3)]
expected = bin(x).count('1') % 2
tests.append((bits, expected))
correct = 0
start = time.perf_counter()
for bits, expected in tests:
```

```
net.reset()
for i, b in enumerate(bits):
net.external_input(i, b)
steps = 0
while not net.is_quiet() and steps < max_steps:
net.step()
steps += 1
if net.neurons[output_idx].state == expected:
correct += 1
end = time.perf_counter()
acc = correct / len(tests)
if acc == 1.0:
return acc, (end - start) * 1e6
else:
return acc, None


def evaluate_adder(net, max_steps=5):
total = len(net.neurons)
if total < 4: # минимум 2 входа + 2 выхода
return 0.0, None
sum_idx = total - 2
carry_idx = total - 1
tests = [(0,0,0,0), (0,1,1,0), (1,0,1,0), (1,1,0,1)]
correct = 0
start = time.perf_counter()
for a, b, exp_sum, exp_carry in tests:
net.reset()
net.external_input(0, a)
net.external_input(1, b)
steps = 0
while not net.is_quiet() and steps < max_steps:
net.step()
steps += 1
if net.neurons[sum_idx].state == exp_sum and net.neurons[carry_idx].state == exp_carry:
correct += 1
end = time.perf_counter()
acc = correct / len(tests)
if acc == 1.0:
return acc, (end - start) * 1e6
else:
return acc, None



# --- Определение задач ---
tasks = [
{
'name': 'xor',
```

```
'n_in': 2,
'n_out': 1,
'evaluate': lambda net: evaluate_bool_func(net, [0,1,1,0]),
'max_steps': 3,
},
{
'name': 'and',
'n_in': 2,
'n_out': 1,
'evaluate': lambda net: evaluate_bool_func(net, [0,0,0,1]),
'max_steps': 3,
},
{
'name': 'or',
'n_in': 2,
'n_out': 1,
'evaluate': lambda net: evaluate_bool_func(net, [0,1,1,1]),
'max_steps': 3,
},
{
'name': 'detector_101',
'n_in': 1,
'n_out': 1,
'evaluate': evaluate_detector_101,
'max_steps': 5,
},
{
'name': 'parity3',
'n_in': 3,
'n_out': 1,
'evaluate': evaluate_parity3,
'max_steps': 5,
},
{
'name': 'adder',
'n_in': 2,
'n_out': 2,
'evaluate': evaluate_adder,
'max_steps': 5,
}
]

# Инициализация хранилища результатов
results = {}
for task in tasks:
results[task['name']] = {
'best_acc': 0.0,
'best_acc_genotype': None,
'best_time': float('inf'),
'best_time_genotype': None
```

```
}

# Параметры поиска
total_attempts = 100000
min_neurons = 3
max_neurons = 15

print(f"Запуск случайного поиска на {total_attempts} попыток")
print("Задачи:", [t['name'] for t in tasks])
print()

start_time_all = time.time()

for attempt in range(1, total_attempts + 1):
# Генерация случайной сети
num_neurons = random.randint(min_neurons, max_neurons)
genotype = random_genotype(num_neurons)
try:
net = build_network_from_genotype(genotype, Network,
{'simple': SimpleNeuron, 'addressable': AddressableNeuron})
except Exception as e:
# Если сеть не построилась, пропускаем
continue

# Тестируем на всех задачах
for task in tasks:
# Проверяем, хватает ли нейронов для размещения входов и выходов
if task['n_in'] + task['n_out'] > num_neurons:
continue # не хватает нейронов, пропускаем задачу для этой сети

# Вызов функции оценки
acc, t_us = task['evaluate'](net)

# Обновление статистики
task_name = task['name']
res = results[task_name]

# Лучшая точность
if acc > res['best_acc']:
res['best_acc'] = acc
res['best_acc_genotype'] = deepcopy(genotype)

# Лучшее время при точности 1.0
if acc == 1.0 and t_us is not None and t_us < res['best_time']:
res['best_time'] = t_us
res['best_time_genotype'] = deepcopy(genotype)

# Прогресс
if attempt % 1000 == 0:
elapsed = time.time() - start_time_all
```

```
print(f"Попытка {attempt}/{total_attempts} ({elapsed:.1f} с)")
# Можно вывести промежуточные лучшие результаты
for task_name, res in results.items():
if res['best_acc'] == 1.0:
print(f" {task_name}: acc=1.0, time={res['best_time']:.3f} мкс")
else:
print(f" {task_name}: best_acc={res['best_acc']:.4f}")

# Сохранение и финальный отчёт
print("\n" + "="*60)
print("ФИНАЛЬНЫЕ РЕЗУЛЬТАТЫ")
print("="*60)

for task in tasks:
name = task['name']
res = results[name]

# Сохраняем лучший по точности (если есть)
if res['best_acc_genotype'] is not None:
acc_filename = f"best_{name}_acc.json"
with open(acc_filename, 'w') as f:
json.dump(res['best_acc_genotype'], f, indent=2)
print(f"{name:15} | точность {res['best_acc']:.4f} | сохранён в {acc_filename}")

# Сохраняем лучший по времени (если есть точность 1.0)
if res['best_time_genotype'] is not None:
time_filename = f"best_{name}_time.json"
with open(time_filename, 'w') as f:
json.dump(res['best_time_genotype'], f, indent=2)
print(f"{name:15} | точность 1.0 | время {res['best_time']:.3f} мкс | сохранён в
{time_filename}")
else:
print(f"{name:15} | решение с точностью 1.0 не найдено")

total_time = time.time() - start_time_all
print(f"\nОбщее время выполнения: {total_time:.2f} с")
```

A.5. Модуль benchmark_best.py

Скрипт для многократной проверки стабильности найденных лучших сетей:

•загружает JSON-файлы с генотипами, показавшими наилучшее время;
•выполняет заданное число прогонов (например, 100 000) для каждой сети, фиксируя точность и время;
•выводит итоговую статистику, подтверждающую детерминизм и отсутствие ошибок.

```python
#!/usr/bin/env python3
import json
import time
import random
from my_neuron import SimpleNeuron, AddressableNeuron
from network import Network
from genotype import build_network_from_genotype

# Импортируем функции оценки из multi_task_random (или продублируем их здесь)
from multi_task_random import (
evaluate_bool_func, evaluate_detector_101,
evaluate_parity3, evaluate_adder
)

# Соответствие между именем задачи и файлом с лучшим генотипом
best_files = {
'xor': 'best_xor_time.json',
'and': 'best_and_time.json',
'or': 'best_or_time.json',
'detector_101': 'best_detector_101_time.json',
'parity3': 'best_parity3_time.json',
'adder': 'best_adder_time.json',
}

# Параметры задач (повторяем из multi_task_random)
tasks = {
'xor': {
'n_in': 2, 'n_out': 1,
'evaluate': lambda net: evaluate_bool_func(net, [0,1,1,0]),
'max_steps': 3
},
'and': {
'n_in': 2, 'n_out': 1,
'evaluate': lambda net: evaluate_bool_func(net, [0,0,0,1]),
'max_steps': 3
},
'or': {
'n_in': 2, 'n_out': 1,
'evaluate': lambda net: evaluate_bool_func(net, [0,1,1,1]),
'max_steps': 3
},
'detector_101': {
'n_in': 1, 'n_out': 1,
'evaluate': evaluate_detector_101,
'max_steps': 5
},
'parity3': {
'n_in': 3, 'n_out': 1,
'evaluate': evaluate_parity3,
'max_steps': 5
```

```
},
'adder': {
'n_in': 2, 'n_out': 2,
'evaluate': evaluate_adder,
'max_steps': 5
}
}

def benchmark_task(task_name, num_runs=10000):
"""Загружает лучший генотип для задачи и прогоняет его num_runs раз."""
# Загрузка генотипа
filename = best_files[task_name]
with open(filename, 'r') as f:
genotype = json.load(f)

# Построение сети
net = build_network_from_genotype(
genotype, Network,
{'simple': SimpleNeuron, 'addressable': AddressableNeuron}
)

# Получаем функцию оценки для этой задачи
task_info = tasks[task_name]
evaluate = task_info['evaluate']
max_steps = task_info['max_steps']

# Прогон
correct_total = 0
start_time = time.perf_counter()

for _ in range(num_runs):
# evaluate возвращает (accuracy, time_us) — нам нужна только точность
acc, _ = evaluate(net)
# Если acc == 1.0, значит все тесты в этом прогоне пройдены
if acc == 1.0:
correct_total += 1
else:
# Если вдруг ошибка, можно сразу вывести и остановиться
print(f"Ошибка на прогоне {_} для задачи {task_name}!")
# Здесь можно сохранить состояние сети для анализа
break

end_time = time.perf_counter()
total_time = end_time - start_time
avg_time_us = (total_time / num_runs) * 1e6

print(f"\n=== {task_name} ===")
print(f"Прогонов: {num_runs}")
print(f"Успешных: {correct_total} / {num_runs} (точность {correct_total/num_runs*100:.6f}%)")
print(f"Общее время: {total_time:.3f} с")
```

```
print(f"Среднее время на прогон: {avg_time_us:.3f} мкс")

if __name__ == "__main__":
# Можно задать количество прогонов через аргумент командной строки
import sys
num_runs = int(sys.argv[1]) if len(sys.argv) > 1 else 10000

for task_name in best_files.keys():
benchmark_task(task_name, num_runs)
```

A.6. Папка best_genotypes

Содержит JSON-файлы с генотипами, достигшими точности 1.0 для каждой задачи. Имена файлов соответствуют шаблону best_<задача>_acc.json (лучший по точности) и best_<задача>_time.json (лучший по времени при точности 1.0). Эти файлы могут быть загружены и использованы для повторения экспериментов или анализа структуры сетей.

Все модули написаны на языке Python версии 3.10 и не требуют установки дополнительных библиотек, кроме стандартных (random, json, time, copy, collections). Для запуска экспериментов достаточно выполнить скрипт multi_task_random.py; для проверки стабильности – benchmark_best.py.